

Important: You have the choice between two different graded projects (A or B). There is no need to go through both of them. Here are the instructions for project A.

NLP Graded Project **A**:

Building and Evaluating a Retrieval-Augmented Generation (RAG) System

Goal of the project

The goal of this project is to design, implement, and evaluate a complete Retrieval-Augmented Generation (RAG) pipeline capable of answering questions based on a closed document collection.

Learning Objectives

By completing this project, you should be able to:

- Build an end-to-end NLP system.
 - Apply information retrieval techniques to textual data.
 - Use dense vector representations (embeddings).
 - Implement and evaluate a Retrieval-Augmented Generation pipeline.
 - Design meaningful experiments and analyze their outcomes.
 - Critically assess the strengths and limitations of your approach.
-

Dataset

Project Instructions: Working with the FinanceBench Dataset

To build and evaluate your RAG system, we will use **FinanceBench**, an industry-standard dataset designed to test LLMs on complex financial documents:

<https://github.com/patronus-ai/financebench>

Instead of manually scraping PDFs and making up questions, you are provided with two JSONL (JSON Lines) files. You can use them programmatically to construct both your vector database and your evaluation pipeline.

1. Understanding Your Data Sources

You will work with two complementary files that act like a relational database. They are linked together by a common key: `doc_name`.

File Name	Purpose	Key Attributes to Extract
<code>financebench_document_information.jsonl</code>	The Catalog: Contains metadata and URLs to download the raw PDF documents.	<code>doc_name</code> (Primary Key), <code>doc_link</code> (URL to the PDF).
<code>financebench_open_source.jsonl</code>	The Ground Truth: Contains the test questions, the expected answers, and the exact source text.	<code>doc_name</code> (Foreign Key), <code>question</code> , <code>answer</code> , <code>evidence_text</code> .

2. Step-by-Step Implementation Guide

To successfully complete the data pipeline phase of your project, you can follow these sequential steps:

Step 1: Automate the PDF Retrieval

- Load `financebench_document_information.jsonl` using Python (e.g., via the `json` module or `pandas`).
- Extract the unique `doc_name` and their corresponding `doc_link`.
- Write a Python script to iterate through these links and download the raw PDF files into a local directory (e.g., `data/raw_pdfs/`).
- *Tip:* Implement error handling in your script. Some links might be slow or require specific headers (like `User-Agent`) to avoid being blocked by SEC EDGAR servers.

Step 2: Build Your Knowledge Base (Ingestion)

- Process the PDFs you just downloaded.

- Apply your chosen PDF parsing and chunking strategies (handling text, tables, and financial layouts).
- Embed the chunks and store them in your Vector Database. Your RAG is now "loaded" and ready to be queried.

Step 3: Run the Evaluation Pipeline

- Load `financebench_open_source.jsonl`.
- Create an evaluation loop: For each item in the file, extract the `question`.
- Pass this `question` to your RAG system and capture the generated response and the retrieved contexts.
- Compare your RAG's output against the ground truth `answer` and the expected `evidence_text` using your chosen evaluation framework (e.g., RAGAS, TruLens, or LLM-as-a-judge).

Your system must answer questions using only the information contained in the provided corpus.

You can choose a different dataset, if that makes you more comfortable.

Project Tasks

1. Corpus Processing

You must preprocess the document collection and prepare it for retrieval.

This includes:

- Text extraction from source documents
- Data cleaning
- Document segmentation (chunking)
- Metadata management

You should justify all design choices and investigate the impact of:

- Chunk size
 - Chunk overlap
 - Document structure preservation
-

2. Retrieval System

Implement a retrieval component capable of identifying the most relevant passages for a given user query.

Documents should be indexed as vector representations and retrieved using similarity search.

3. Answer Generation

Using the retrieved passages, generate a natural-language answer to the user's question.

Constraints

- Only free and locally executable models may be used.
- The generated answer should include references to the retrieved passages used as evidence.

Examples of suitable models include:

- Llama
 - Mistral
 - Qwen
 - Phi
-

5. Experimental Analysis

A major objective of this project is to understand which design decisions affect system performance.

You must compare several configurations.

Examples include:

- Different embedding models
- Different chunk sizes
- Different values of k (number of retrieved passages)
- Different generation models

The focus should be on rigorous experimentation and analysis rather than simply achieving the highest score.

Deliverables

1. Source Code

Your submission must include:

- Well-organized and documented code
 - Installation instructions
 - A README explaining how to reproduce the experiments
-

2. Scientific Report

The report should contain:

1. Introduction
2. Corpus Description
3. Methodology
4. Experimental Setup
5. Results
6. Error Analysis
7. Discussion
8. Conclusion

Figures, tables, and plots are encouraged.

Technical Requirements

You can use, for example, the following tools and libraries

- Python
 - PyTorch or Tensorflow
 - Hugging Face Transformers
 - Sentence Transformers
 - FAISS
 - Chroma
 - LangChain
 - LlamaIndex
-

Deadline:

The project is due two weeks after the last course (Course 5). No late submission will be accepted unless you get an explicit permission from me. You can work on this project in a

group of **one**, **two** or **three** students. It is mandatory to complete either project A or project B. Not completing either will automatically result in getting the lowest grade.